CREATE

D8

```
<?php

// Procedural - There is no entity_save() anymore
$node = entity_create('node', array('title' => 'My Title', 'type' => 'page'));
$node->save();

// Static Call
$node = Node::create(['type' => 'page']);
$node->set('title', 'My Title');
$node->save();

// Entity Manager
$info = array('type' => 'page', 'title' => 'My Title');
$node = \Drupal::entityTypeManager()->getStorage('node')->create($info);
$node->save();
```

D7

```
<?php
// Using Drupal Core.
global $user;
$node = new stdClass();
$node->title = "My Title";
$node->type = "basic_page";
node_object_prepare($node);
$node->language = LANGUAGE_NONE;
$node->uid = $user->uid;
$node->status = 1;
$node->promote = 0;
$node = node_submit($node);
node_save($node);

// Using Entity API
$node = entity_create('node', array('type' => 'basic_page'));
$entity = entity_metadata_wrapper('node',$node);
$entity->title->set("My Title");
$entity->save();
```

READ

D8

```
<?php
```

```
// Procedural Calls
$node = entity_load('node', 1);
$nodes = entity_load_multiple('node', array(1, 2, 3));

// Static Calls
$nid = 1;
$node = Node::load($nid);

$nids = array(1, 2, 3);
$nodes = Node::loadMultiple($nids);

// Using the Entity Manager
$storage = \Drupal::entityManager()->getStorage('node');
$node = $storage->load(1);
$nodes = $storage->loadMultiple(array(1, 2, 3));
```

D7

```
<?php

// Using Drupal Core Functions
$nid = 1;
$node = node_load($nid);

// Both code below does the same thing
$nids = array(1, 2, 3);
$nodes = node_load_multiple($nodes);
$node_entities = entity_load('node', $nids);

// Using the Entity API
$node_entity = entity_load_single('node', $nid);

// Loading taxonomy
$term = taxonomy_load_term(25);

// Loading a user
$user = user_load(1);
```

UPDATE
D8

```
<?php
// Procedural there is no entity_save
$node = entity_load_single('node', 1);
$node->save()
```

```
// Static calls
$nid = 1;
$node = Node::load($nid);
$node->set('title', 'My Awesome New Title');
$node->set('field_my_field', 'My New field');
$node->save();

// Storage Manager
$storage = \Drupal::entityManager()->getStorage('node');
$node = $storage->load(123);
$node->set('field_my_field', 'value');
$node->save();
```

D7

```
<?php
// Drupal core
$nid = 1;
$node = node_load($nid);
$node->title = "My New Title";
node_save($node);

// Entity API
$nid = 1;
$node_entity = entity_load_single('node', $nid);
$node_wrapper = entity_metadata_wrapper('node', $node_entity);
$node_wrapper->title->set('My Title');
$node_wrapper->save();
```

DELETE

D8

```
<?php
// Procedural - there is no entity_delete
entity_delete_multiple('node', array(1));

// Static call
$nid = 1;
$node = Node::load($nid);
$node->delete();

// DELETE
$storage = \Drupal::entityManager()->getStorage('node');
$node = $storage->load(123);
```

```
$node->delete();
```

D7

```
<?php
$nid = 1;
node_delete($nid);

// Entity API
$nid = 1;
$node_entity = entity_load_single('node', $nid);
$node_wrapper = entity_metadata_wrapper('node', $node_entity);
$node_wrapper->delete();
```

Accessing Entity Values

```
// Drupal 7 Core
$node = node_load(1);
$title = $node->title;
$field = $node->field_my_field[LANGUAGE_NONE][0]['value'];

// Entity Metadata Wrapper
$node = node_load('node', $node);
$wrapper = entity_metadata_wrapper('node', $node);
$title = $wrapper->title->value();
$field = $wrapper->field_my_field->value();

// D8
$node = Node::load(1);
$node_array = $node->toArray();

// Various ways to get title
$title = $node->get('title')->value;
$title = $node->get('title')->getString();

// Referenced Entity
$node->get('field_my_reference')->referencedEntities();

// Chaining Methods
$node->get('field_my_reference')->referencedEntities()[0]->get('name')->getString()
```

EntityQuery
D8

```
<?php
```

```
$nids = \Drupal::entityQuery('node')
      ->condition('status', 1)
      ->condition('type', 'page')
      ->condition('field_my_field', 'My Value')
      ->execute();

if (!empty($nids)) {
    foreach ($nids as $nid) {
        $basic_page = Node::load($nid);

        // Do Sweet Stuff With Your Loaded Node.
    }
}
```

EntityFieldQuery
D7

```
<?php
$query = new EntityFieldQuery();
$query->entityCondition('entity_type', 'node')
  ->entityCondition('bundle', 'article')
  ->propertyCondition('status', NODE_PUBLISHED)
  ->fieldCondition('field_news_types', 'value', 'spotlight', '=');

$result = $query->execute();

if (isset($result['node'])) {
  $news_items_nids = array_keys($result['node']);
  $news_items = entity_load('node', $news_items_nids);
}
```

Entity Hooks

Loading Hooks

hook_entity_load
hook_ENTITY_TYPE_load
hook_entity_storage_load

Pre Render Hooks

hook_entity_prepare_view
hook_entity_view
hook_ENTITY_TYPE_view
hook_entity_view_alter

Entity Hooks

CRUD Hooks

hook_entity_create hook_ENTITY_TYPE_create hook_entity_insert hook_ENTITY_TYPE_insert hook_entity_update hook_ENTITY_TYPE_update hook_entity_presave hook_ENTITY_TYPE_presave hook_entity_predelete hook_ENTITY_TYPE_predelete	hook_entity_delete hook_ENTITY_TYPE_delete hook_entity_revision_delete hook_ENTITY_TYPE_revision_delete hook_entity_prepare_form hook_ENTITY_TYPE_prepare_form

Entity Hooks

Translation Hooks

hook_entity_translation_create
hook_ENTITY_TYPE_translation_create
hook_entity_translation_insert
hook_ENTITY_TYPE_translation_insert
hook_entity_translation_delete
hook_ENTITY_TYPE_translation_delete

Entity Hooks

Access Hooks

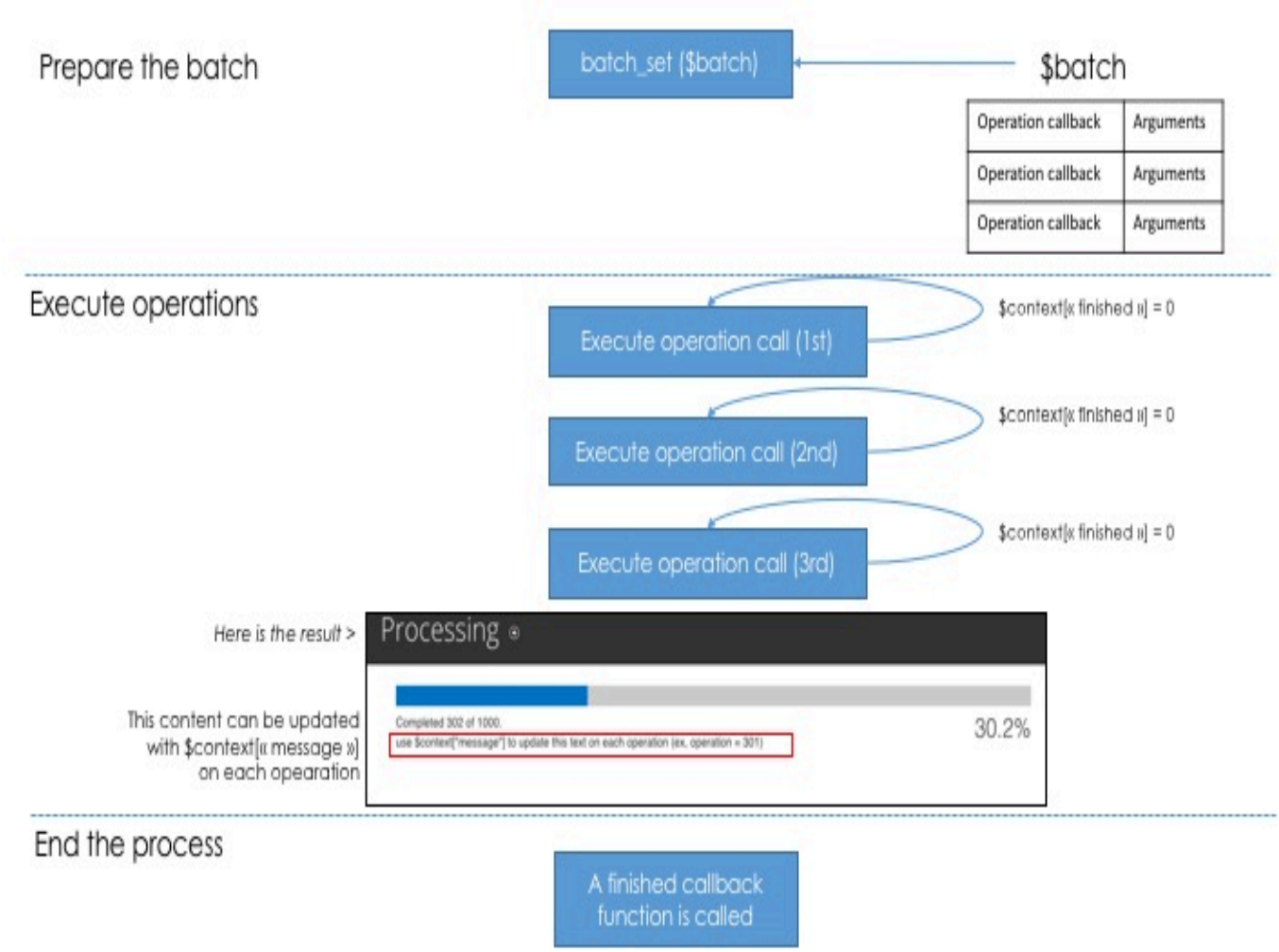
hook_entity_access
hook_ENTITY_TYPE_access
hook_entity_create_access
hook_ENTITY_TYPE_create_access

Implement Custom REST API

web/modules/custom/custom_apis/src/Plugin/rest/resource/GetArticles.php

```php
<?php

namespace Drupal\custom_apis\Plugin\rest\resource;

use Drupal\rest\Plugin\ResourceBase;
use Drupal\rest\ResourceResponse;
use Drupal\node\Entity\Node;
use Drupal\taxonomy\Entity\Term;

/**
 * Provides a Demo Resource
 *
 * @RestResource(
 *   id = "articles",
 *   label = @Translation("Articles listing"),
 *   uri_paths = {
 *     "canonical" = "/get/articles",
 *     "create" = "add/articles"
 *   }
 * )
 */
class GetArticles extends ResourceBase
{
  /**
   * Responds to entity GET requests.
   * @return \Drupal\rest\ResourceResponse
   */
  public function get()
  {
    try {
      $nids = \Drupal::entityQuery('node')->condition('type', 'article')->execute();
      $nodes =  \Drupal\node\Entity\Node::loadMultiple($nids);
      $response = $this->processNodes($nodes);
    return new ResourceResponse($response);
    } catch (EntityStorageException $e) {
      \Drupal::logger('custom-rest')->error($e->getMessage());
    }
  }
```

```
  /**
   * Get articles
   */
  private function processNodes($nodes)
  {
    $output = [];
    foreach ($nodes as $key => $node) {
      $output[$key]['title'] = $node->get('title')->getValue();
      $output[$key]['node_id'] = $node->get('nid')->getValue();
      $output[$key]['description'] = $node->get('body')->getValue();
    }
    return $output;
  }

  /**
   * Post api
   */
 public function post($data){
   try {

     $new_term = Term::create([
       'name' => $data['title'],
       'vid' => $data['type'],
     ]);
     $new_term->save();

     return new ResourceResponse('Term created successfully in '. $data['type']);
   } catch (EntityStorageException $e) {
     \Drupal::logger('custom-rest')->error($e->getMessage());
   }
 }
}
```



Adding Regions to Your Info File

Start by declaring any new regions in your THEMENAME.info.yml file. Regions are declared as children of the `regions` key like so:

```
regions:
  header: 'Header'
  content: 'Content'
  footer: 'Footer'
```

Region keys should be alphanumeric and can include underscores (_). Keys should begin with a letter. The key is the machine name (which you use in code) and the value a human readable version displayed in the admin UI.

Adding Regions to Your Templates

In order for regions to display any content placed into them, you'll need to make sure your new regions are also added to your `page.html.twig` file. Regions will be represented as Twig variables whose name corresponds with the key used in your THEMENAME.info.yml file with the string `page.` prepended.

Example:

```
header: 'Header'
```

...will become:

```
{{ page.header }}
```

These behave like any other Twig variable and may be wrapped in whatever markup makes sense for your use case.

(The syntax for the default hidden regions is different, see below.)

Default Regions

See the page.html.twig documentation for a list of default regions.

```
regions:
  header: 'Header'
  primary_menu: 'Primary menu'
  secondary_menu: 'Secondary menu'
  highlighted: 'Highlighted'
  help: 'Help'
  content: 'Content'
  sidebar_first: 'Left sidebar'
  sidebar_second: 'Right sidebar'
  footer: 'Footer'
  breadcrumb: 'Breadcrumb'
  page_top: 'Page top'
  page_bottom: 'Page bottom'
```

The default regions are defined in `core/lib/Drupal/Core/Extension/ThemeExtensionList.php`

If your theme doesn't declare any regions, Drupal will assume this set of defaults.

These regions correspond with what the default `core/modules/system/templates/page.html.twig` file expects, as well as two hidden regions, `page_top`, and `page_bottom` - you don't need to declare these final two if you override the defaults, however the `{{ page_top }}` and `{{ page_bottom }}` Twig variables should be retained in the `html.html.twig` template.

Drupal Custom Theme Development

STEP 1 : First, we need to create a custom theme under ‘web/themes/custom’ folder. We will name the folder as custom_theme.

STEP 2 : Next, we will need to create an info.yml file. We need to specify the basic keys for it. Let us specify it over here -

CODE:
```
name: Custom Theme
type: theme
description: 'Custom Theme for My Website.'
package: Other
core_version_requirement: ^8 || ^9
```

STEP 3 : Now, let’s create a libraries.yml file to specify all the libraries we need (CSS AND JS) for our custom Drupal 9 theme. We will also create CSS and JS directory and its files to link it in here. We are going to name the library as global-styling.

`CODE:`
```
global-styling:
  version: 1.x
  js:
    js/script.js: {}
  css:
    theme:
      css/style.css: {}
```

STEP 4 : After creating the libraries.yml file, we need to link it to our theme. For this, we are going to add it in the info.yml file which will then apply it to the whole theme.

`CODE:`

```
libraries:
- custom_theme/global-styling
```

So, the key will be libraries and path will be the theme name - ‘custom_theme’ / library name - ‘global-styling’.

STEP 5 : Next, we need to inherit the ‘Base Theme’.In our case, we will inherit the ‘classy’ theme which is a Drupal core theme. So, the key will be the base theme in info.yml.

CODE:

```
base theme: classy
```

STEP 6 : Now, we will define the‘regions’ for our theme. In info.yml, we have to define it under the ‘regions’ key.

CODE:

```
regions:
branding: Branding
navigation: Main navigation
search: Search
featured: Featured
content: Content
right_sidebar: Right sidebar
footer_first: Footer First
footer_second: Footer Second
footer_third: Footer Third
footer_bottom: Footer Bottom
```

Under ‘regions’ key, you can define your regions for the custom Drupal theme. Here,

branding: Is the id of the region which should be lowercase letters.

Branding: Is the name of the region which can be uppercase letters.

custom_theme.info.yml

web > themes > custom > custom_theme > custom_theme.info.yml

```yaml
name: Custom Theme
type: theme
description: 'Custom Theme for My Website.'
package: Other
core_version_requirement: ^8 || ^9
libraries:
  - custom_theme/global-styling

base theme: classy
regions:
  branding: Branding
  navigation: Main navigation
  search: Search
  featured: Featured
  content: Content
  right_sidebar: Right sidebar
  footer_first: Footer First
  footer_second: Footer Second
  footer_third: Footer Third
  footer_bottom: Footer Bottom
```

Defining 'regions'

Under ‘regions’ key, you can define your regions for the custom Drupal theme. Here,
branding: Is the id of the region which should be lowercase letters.
Branding: Is the name of the region which can be uppercase letters.

STEP 7 : After we have defined our regions for our custom Drupal theme, we need to override page.html.twig to grab our ‘regions’instead of the classy theme’s. We will create templates/system directory under which we will create the page.html.twig.

Override the page.html.twig

CODE:

```
<header aria-label="Site header" class="header" id="header" role="banner">
    {{ page.branding }}
    {{ page.navigation }}
    {{ page.search }}
</header>
<section class="featured" id="featured" role="complementary">
   {{ page.featured }}
</section>
<section class="main" id="main">
```

```
  <main aria-label="Site main content" class="content" id="content" role="main">
    {{ page.content }}
  </main>
  <aside class="right--sidebar" id="sidebar" role="complementary">
    {{ page.right_sidebar }}
  </aside>
</section>
<footer aria-label="Site footer" class="footer" id="footer" role="contentinfo">
  <div class="footer--top">
    {{ page.footer_first }}
    {{ page.footer_second }}
    {{ page.footer_third }}
  </div>
  <div class="footer--bottom">
      {{ page.footer_bottom }}
  </div>
</footer>
```

In page.html.twig, we will create the HTML structure for our regions. As you see in {{ page.branding }} – Here,

page - Is the key to render ‘regions’ in the page

branding- Is the region which we have defined in info.yml file.

So now, we have created our regions and rendered it on the page.

Step 8 : Go to Appearance in your Drupal site. You can see your custom Drupal theme present in the Uninstalled themes section.

Step 9 : You are now almost done with theming in Drupal 9! Next, you need to apply styles in the CSS for each region as per your design. We will use CSS in this case; You can even use SCSS if you’d like. Just inspect the branding region - you should see the region class and then add the CSS to that class.

Add CSS in style.css as per your requirement.

```
.header{
 display: flex;
 justify-content: space-between;
 padding: 10px;
}
.header.region {
```

```
 padding: 5px;
}
.header.region-branding {
 flex: 0 1 20%;
}
.header.region-navigation {
 flex: 0 1 50%;
}
.header.region-search {
 flex: 0 1 30%;
}
.region.block-region {
 padding: 15px;
}
.featured{
 padding: 40px 20px;
 background-color: indianred;
}
.main{
 padding: 50px 0;
 display: flex;
 justify-content: space-between;
}
.main.content {
 flex: 0 1 65%;
}
.main.right--sidebar {
 flex: 0 1 30%;
}
.footer--top {
 display: flex;
 justify-content: space-between;
 padding: 10px;
}
.footer--top .region {
 padding: 5px;
}
.region-footer-first, .region-footer-second, .region-footer-third {
 flex: 0 1 30%;
}
```

Determine repeat characters in a php string

```
$string = "aabbbccddd";
$array = array();
foreach (count_chars($string, 1) as $i => $val) {
        $count = chr($i);
        $array[] = $val ."-". $count;
}
print_r($array);
```

Output: `Array ( [0] => 2-a [1] => 3-b [2] => 2-c [3] => 3-d )`

Swap two variable values without using the 3rd variable.

$a = 3; $b = 8;

$a = $a + $b;

$b = $a - $b;

$a = $a - $b;

Output: $a = 8 & $b = 3;

Prime Numbers program: Number which is divisible by itself like

Prime Numbers from 1 to 100

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97.

Print this series using programmatically.

```
function IsPrime($n)
{
 for($x=2; $x<$n; $x++)
   {
      if($n %$x ==0)
           {
             return 0;
            }
    }
  return 1;
   }
$a = IsPrime(3);
if ($a==0)
echo 'This is not a Prime Number.....'."\n";
```

```
else
echo 'This is a Prime Number..'."\n";
```

The Fibonacci Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233…

$F_n = F_{n-1} + F_{n-2}$

With boundary conditions: $F_0 = 0$ and $F_1 = 1$
https://www.alphacodingskills.com/php/pages/php-program-to-check-prime-number.php

Frequently Asked Questions

Why PHP 7 is faster?
PHP 7 runs on the PHPNG engine (or Zend Engine 3.0) that speeds up PHP apps more than the previous PHP interpreter (Zend Engine 2.0). Thanks to PHPNG, your applications will perform 2x faster and has 50% better memory consumption than PHP 5.6. It enables you to serve more concurrent users without adding any new hardware.

What are the most important advantages of using PHP 7 over PHP 5?
Upgrading to PHP7 not only guarantees fast website loading but is also far efficient scripting that runs effortlessly every time users access the website. Hence, if businesses choose to upgrade their websites from PHP5 to PHP7, it would help their businesses to grow.

Is PHP 5 still supported?
No. December 31, 2018, indicates the official end of security support for PHP 5.6. It is advised that you should not use PHP 5.6 (or any version of PHP 5) in a production environment. There are a number of deprecated and obsolete functions and libraries that should no longer be used, considering security reasons.

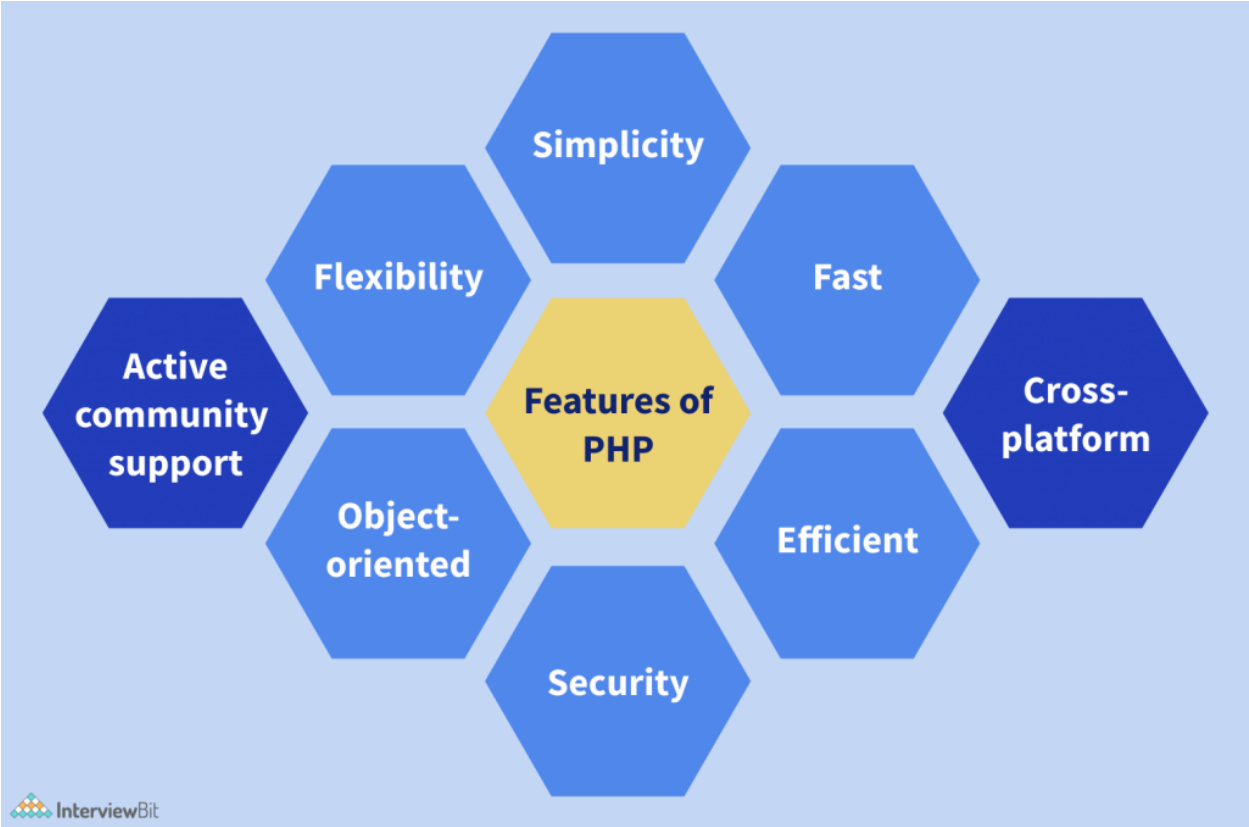
Is PHP 7 more secure?
Each release branch of PHP is fully supported for 2 years from its first stable release. During this stage, errors and security issues that have been reported are resolved and are released in regular point releases. PHP 7 offers better security improvements as

compared to PHP 5, which includes a filtered un-serialized function and a set of functions to easily get cryptographically secure random numbers.

Is PHP 5 a security risk?
By the end of 2018, PHP stopped security updates and support for some of its earlier versions. This means it will expose hundreds of millions of websites to serious danger in terms of user details stolen, sites hacked, and massive fines. You need to update, and use systems that permit you to deploy only new versions of PHP by default.

Features of PHP

Now that we have a basic understanding of PHP, let us see some of the most striking features of PHP.



Features of PHP

Simplicity

PHP is particularly famous for its simplicity. It is organized and easy to learn. Even beginners won't face any hard time learning and using PHP. It is a very well-organized programming language, and it comes with a lot of pre-defined functions, which makes the task of the programmer easy. There is no need to include libraries in PHP like C. With a lot of pre-defined functions, PHP is easy to optimize as well.

Flexibility

PHP scripts can run on any device- mobile, tablet, or PC. It is very compatible with various databases. It can be easily embedded and integrated into HTML, XML, and JavaScript. Likewise, it is also compatible with almost all servers used today like Apache, IIS, etc.

Objective oriented

PHP supports object-oriented programming features like data encapsulation, inheritance, abstraction, polymorphism, etc. The Object-oriented programming feature was added in PHP5. This feature helps in building complex reusable web pages and makes PHP comparable to powerful object-oriented languages like Java and Python.

Interpreted language

PHP is an interpreted language, which means there is no need for compilation. Interpreters run through a program line by line and execute the code. Since interpreters execute source code themselves, the code becomes platform-independent. Some other benefits of interpreted language include dynamic typing and short executable program size.

Efficient

PHP is a versatile, reliable, and efficient programming language. The memory management of PHP is very efficient. Great session management, eliminating unnecessary memory allocation, are some of the features that make PHP efficient.

Fast Performance

PHP scrips are usually faster than other scripting languages. Users can load their web pages faster, and they love it. PHP code runs faster than most of programming languages because it runs in its own memory space. Not only that, but its connection with databases is also fast and efficient.

Case-sensitive

PHP is a partially case-sensitive language. Although functions names are not case-sensitive, other things in PHP are case-sensitive. The following things in PHP are case-sensitive:

- Variable names
- Constructs (if, if-else, if-elseif, while, do-while)
- Keywords (such as true and false)
- User-defined functions and class names

Security

PHP has many pre-defined functions for data encryption. Users can also use third-party applications for security. Security and flexibility are often contrasting features, but PHP somehow manages to offer them both, and that's great. PHP is designed specifically to be a more secure language for writing CGI (Computer-generated Imagery) programs. Security algorithms such as Sha1 (secure Hash algorithm 1) and MD5(Message digest 5) are used to encrypt the strings in PHP. Filter_var and strip_tags functions help to keep the environment more secure and safe for users.

PHP 8 Improvements and New Features

Apart from JIT, we can expect many features and improvements with PHP 8. The following list is our handpicked selection of the upcoming additions and changes that should make PHP more reliable and efficient.

- Constructor Property Promotion
- Validation for Abstract Trait Methods
- Incompatible Method Signatures
- Arrays Starting With a Negative Index
- Union Types 2.0
- Consistent Type Errors for Internal Functions
- throw Expression
- Weak Maps
- Trailing Comma in Parameter List
- Allow ::class syntax on objects
- Attributes v2
- Named Arguments
- Nullsafe Operator
- Saner String to Number Comparisons
- Saner Numeric Strings
- Match Expression v2
- Stricter Type Checks for Arithmetic/Bitwise Operators

MyISAM vs InnoDB

Now that we have a basic understanding of MyISAM and InnoDB, let's compare them on various factors:

- MyISAM vs InnoDB: Storage Engine Type
- MyISAM vs InnoDB: Locking
- MyISAM vs InnoDB: Foreign Keys
- MyISAM vs InnoDB: ACID Properties
- MyISAM vs InnoDB: Performance
- MyISAM vs InnoDB: Reliability
- MyISAM vs InnoDB: Caching and Indexing

1. MyISAM vs InnoDBStorage: Engine Type

MyISAM is a non-transactional storage type, and any write option needs to be rolled back manually (if needed).

InnoDB is a transaction storage type that automatically rollbacks the writes if they are not completed.

2. MyISAM vs InnoDBStorage: Locking

Locking is the mechanism in MySQL that prevents two users from modifying the duplicate rows at the same time by locking the row. The users can't change the table when locking is enabled.

MyISAM uses the default method of table locking and allows a single session to modify the table. This means only one user at a time can alter the table. If another user tries to change the table, they will get a message saying that it is locked. The table locking method is helpful for read-only databases as it doesn't require a lot of memory.

InnoDB uses row-level locking of the table. This method supports multiple sessions on the same row by only locking the rows in the modification process. Row-locking is helpful for databases that have multiple users.

The only disadvantage of row-level locking is that it consumes a lot of memory, and querying and modifying data takes time.

3. MyISAM vs InnoDBStorage: Foreign Keys

A foreign key is a column in one table that links the data to another table. It prevents users from adding records that destroy the link between two tables.

MyISAM doesn't support the Foreign key option.

InnoDB supports the Foreign Key option.

4. MyISAM vs InnoDBStorage: ACID Properties

ACID stands for Atomicity, Consistency, Isolation, and Durability. MyISAM doesn't support ACID properties whereas InnoDB supports ACID properties.

5. MyISAM vs InnoDBStorage: Performance

InnoDB supports transactional properties, i.e. rollbacks and commits, and has a higher speed of writing. The performance of InnoDB for large volumes of data is better as compared to MyISAM.

MyISAM doesn't support transactional properties and is faster to read. As compared to InnoDB, the performance for a high volume of data is less.

6. MyISAM vs InnoDBStorage: Reliability

InnoDB uses a transactional log to log every operation and hence gives reliable operations. Thus, in case of failure, data can be recovered quickly by using those logs.

MyISAM offers no data integrity; hardware failures and canceled operations can cause data to become corrupt.

7. MyISAM vs InnoDBStorage: Caching and Indexing

InnoDB supports a large pool of buffers that caches both data and indexes. However, there is no support for a Full-text search.

The **MyISAM** key buffer is only meant for indexes, and a full-text search is supported in MyISAM.

Conclusion

This article listed the critical differences between InnoDB and MyISAM by comparing them against critical factors. It is important to be aware of the properties of each database because it becomes easier to make a choice. You should be aware of the similarities they possess and also what differentiates them. Hopefully, after reading this article you have a better understanding of InnoDB and MyISAM.

PHP 8 Improvements and New Features

Apart from JIT, we can expect many features and improvements with PHP 8. The following list is our handpicked selection of the upcoming additions and changes that should make PHP more reliable and efficient.

- Constructor Property Promotion
- Validation for Abstract Trait Methods
- Incompatible Method Signatures
- Arrays Starting With a Negative Index
- Union Types 2.0
- Consistent Type Errors for Internal Functions
- throw Expression
- Weak Maps
- Trailing Comma in Parameter List
- Allow ::class syntax on objects
- Attributes v2
- Named Arguments
- Nullsafe Operator
- Saner String to Number Comparisons
- Saner Numeric Strings
- Match Expression v2
- Stricter Type Checks for Arithmetic/Bitwise Operators